Creating Embedding Spaces of Short Form Messages for Authorship Verification

Alex Oshima
UC San Diego
aoshima@ucsd.edu

Akhil Pillai
UC San Diego
avpillai@ucsd.edu

Abstract

Authorship verification is typically performed on very long texts, where there are thousands, if not millions, of tokens in which the author's writing style, intent, and diction can be properly analyzed and compared. In practice, whether due to a lack of data or compute power, we often need to work with short texts, for which a bespoke model is desired. To address this, we implement both a deep learning-based approach and a statistical learning-based approach specializing in short text analysis, and compare the two in terms of accuracy, training time, and testing time. The deep learning architecture is a biBERT Siamese model that compares SVM-enhanced representations of each text via distance, and the statistical learning approach uses an SVM ensemble to predict the most likely author of each text, optionally comparing either the predicted label or the probability distribution. We find that the deep learning approach provides much better accuracy without a significant increase in prediction time, while the statistical learning approach takes significantly less time to train.

1. Introduction

Thomas Kyd was a contemporary of Shakespeare, and Mr. Brooks was my 11th grade English teacher. You probably have not seen the writing styles of either individual (unless you are Mr. Brooks—in which case, humor me), but I bet you could still discern the differences between their writing styles. This ability to generalize from specifically trained tasks to unseen information without simply consuming enormous sets of data is one of the major challenges that machine learning models face today. In this paper, we attempt to overcome that hurdle in the field of authorship verification. Authorship verification is the task of taking two input texts and determining whether or not they were written by the same author. Specifically, we examine authorship verification in the context of short-form messages. With social media and other platforms increasingly focused on short-form content, this has become an imperative task.

1.1. Literature Review

The problem of authorship verification is well defined and thoroughly explored. It involves determining, given two inputs, whether those inputs originate from the same source (author). In our case, we are specifically examining text inputs, and more precisely, short-form messages. In the digital world, many works are shifting toward short-form content, such as tweets or comments. These sentence-long inputs are often difficult to attribute to a specific author. This raises our first question: how can authorship verification techniques be applied to modern short-form content?

Like many fields related to natural language processing, large language models (LLMs) and transformer-based architectures have become predominant [3]. However, the challenge with LLM approaches is that they are often computationally expensive and slower compared to specialized models [3]. This begs the question of what lighter-weight models can be leveraged for this task. Statistical models, such as support vector machines (SVMs), are much easier and faster to train than their deep learning counterparts. Combined with traditional NLP methods like n-grams or TF-IDF, SVMs have found success in authorship verification [5]. This outlines our first approach of using SVMs.

Nevertheless, these approaches do not reach the same accuracy as transformer-based architectures [1]. The current state-of-the-art approach involves specially training encoders and neural networks to achieve accuracy in the high 90s [4]. However, this process of extensive fine-tuning diminishes the generalizability of these models. This leads us to our final research question: how can we develop a generalizable authorship verification model that works on authors the model has never seen before?

1.2. Research Hypothesis

When examining the problem of authorship verification, we consider three main questions: How do current techniques perform on short-form messages? Can non-deep learning approaches achieve performance comparable to that of deep learning techniques in this context? And are these models generalizable to unknown authors?

We believe that while statistical models hold value, it

will be especially challenging for them to work effectively in the context of short-form messages, where the input can be as short as one or two words. By leveraging an encoder such as BERT and performing fine-tuning, we believe we can create an authorship verification model that is capable of generalizing to unknown authors.

2. Methodology

2.1. Dataset Selection

We chose to use the Reddit Pushshift dataset for this project. The Pushshift dataset contains *every single* Reddit comment, including deleted ones and comments from accounts that have been deleted or banned. We selected this dataset for two reasons. First, the Pushshift dataset is extremely comprehensive. As of Q4 2024, Reddit has over 100 million active users ([2]). Consequently, we used comments from December 2024, which alone accounted for nearly 300 million comments—more than enough to perform experiments even after processing and cleaning. Second, as is typical for a social media forum, Reddit comments are extremely short compared to the longer texts (such as books and plays) commonly used for authorship verification. The comments in our processed dataset had a median length of 7 words and a mean length of 22.5 words, with the longest comment in the dataset being a mere 1338 words. For these reasons, we found the December 2024 Pushshift dataset to be a suitable source for training data.

2.2. Preprocessing Methodologies

Since we are training on shorter data, each author needs more training comments so that correlations and writing styles can be learned. To start, we filtered the dataset to include only comments from the top 300 most prolific commenters in December 2024. We identified another issue: many of the most prolific accounts are bots, and including them would significantly skew the results, likely affecting the model's ability to generalize to Reddit comments from human users. This outcome is undesirable because we seek to perform both regular authorship verification and *zero-shot* authorship verification with the models, where the input comments may or may not be from users the model was trained on. To combat this, we first removed all comments from `u/AutoModerator`, which alone accounted for nearly 10 million (almost 3%) of the entire preprocessing database. We then removed all comments from accounts whose usernames included the word `bot` (case-insensitively), as many Reddit bots have that word in their username. Although there could be more bots in the dataset, such as those providing AI-generated responses to comments or posts, filtering these out is a project of similar complexity to this one. Removing them would not significantly affect the results since these AI-generated comments are modeled to be very similar to genuine human comments. Thus, we stopped filtering at this point. Our final filtered database contains around 1.2 million comments from 242 different authors. Lastly, each comment has 74 different metrics tracked, of which only four are relevant to authorship identification besides the author: the comment's body, the comment's creation time, the comment's score (upvotes), and the subreddit in which the comment was posted. With all this processing, the December 2024 Pushshift comment dataset was reduced from a massive 600 GB to a much more manageable 500 MB.

2.3. Model Creation

2.3.1 Statistical Learning Approach

For the statistical learning approach, we implemented a method similar to that used by [5], where an SVM was trained on each class and then ensembled together to perform predictions. However, unlike [5], we chose to serialize the comment bodies using TF-IDF instead of n-grams. We selected this approach because n-grams are extremely unfriendly to zero-shot verification: the model will fail if it encounters word pairs that have not been tokenized into an n-gram. On the other hand, TF-IDF simply assigns a term frequency of 0 for unseen words while still performing optimally with the information it does recognize. The model architecture is as follows:

Training: Train an ensemble of SVMs that each yield a probability for a text belonging to one author's class, and use the most confident prediction as the model's overall prediction.

Prediction:

1. Take in two comments, and use the TF-IDF vectorizer formulated from the training data to convert them both into vector representations of 1500 features. (1500 was chosen for the fixed-length representation of variable-length comments, as the longest comment is 1338 words long. Thus, 1500 features should capture all the data in the comment without adding much additional memory overhead.)

2. Pass the vector representation of each comment, along with heuristics such as comment creation time and score, to the model. Retrieve the most confident label predicted by each model.

3. Conclude that the comments are from the same author if these two labels are identical, and that they are from different authors if the labels differ.

Thus, we train our model to perform *authorship attribution* and use the results of those attributions to perform *authorship verification*.

2.3.2 Deep Learning Approach

Architecture Design

The architectural design is centered on leveraging the success that transformers have achieved in various tasks, particularly in natural language processing (NLP). Rather than performing specialized feature engineering on text and other dataset information, we propose using a BERT encoder to accomplish this task. By passing the input texts through the encoder and subsequently fine-tuning its output, we aim to create an embedding feature space where messages from different authors are well separated, while messages from the same author remain in close proximity. This design should enable the SVM to more effectively discriminate between messages.

Dataflow Architecture

The dataflow architecture involves processing both input texts through the encoder, after which the output is passed through a fully connected network. This network incorporates a residual layer that reintegrates the BERT encoder's output back into the network. The final output of this tower constitutes the embedding. This is a Siamese network, so both inputs are passed through the same embedding towers and then compared. For efficiency, they are actually concatenated together before going through BERT and then split.

Verification Task Approaches

From this point, we evaluate multiple methods for incorporating the information into the SVM and performing the verification task.

Similarity Score Method:

The first approach involves feeding the similarity score of the embeddings into the SVM, which then determines the output. This method constrains the SVM's capacity because it relies solely on a similarity value and presupposes a well-trained embedding space. Essentially, the SVM is reduced to determining an appropriate threshold for the similarity score.

Prediction First:

The next two methods grant the SVM greater flexibility by providing the full embeddings as input. In these approaches, the SVM outputs the author classification (from the set of authors on which it was trained); these classifications are subsequently used to ascertain whether the input texts originate from the same author, either directly (if the classified author is identical for both inputs) or by considering the probabilities associated with each author. This procedure effectively enables the SVM to transform the higher-dimensional embedding space into a lower-dimensional author classification space, where each author corresponds to a distinct writing style and higher probabilities indicate that the input text's writing style closely aligns with that author.

Training

The training process consisted of a loop that ran for 4 epochs over 16,000 training examples. Of these, 8,000 were positive examples (pairs of input messages from the same author) and 8,000 were negative examples (pairs of input messages from different authors). After each epoch, accuracy was evaluated on both the set of known authors (from which the embedding space was constructed) and on a set of completely separate authors.

Contrastive Loss Function

The loss function used to train the embedding space is a contrastive loss defined as follows:

```
output1_norm = F.normalize(embedding1, p=2, dim=1)
output2_norm = F.normalize(embedding2, p=2, dim=1)

cosine_sim = F.cosine_similarity(output1_norm,
                output2_norm, dim=1)

loss_similar = (1 - cosine_sim) * label

# Increase the weight for dissimilar pairs
loss_dissimilar = 2.0 * F.relu(cosine_sim - margin)
                    * (1 - label)

loss = torch.mean(loss_similar + loss_dissimilar)
```

In this formulation, dissimilar pairs are weighted by a factor of 2 more than similar pairs. This weighting is applied because the primary objective of the embedding space is to create a more effective feature space for the SVM, rather than solely clustering messages from the same author.

Model Fine-Tuning and Optimization

To enhance the model's ability to distinguish between messages, the top four layers of BERT were reinitialized. This adjustment facilitates better fine-tuning for separating messages from different authors. Additionally, the ADAM optimizer was employed to improve the training process.

The training loss was recorded for each epoch, as well as cumulatively across all epochs, and was graphed at the end of each training epoch to monitor the model's performance over time.

3. Experiments

3.1. Statistical Learning Approach

3.1.1 Experimentation Explanation

In the statistical approach, we trained three models:

1. One trained on 5 authors,
2. One trained on 20 authors, and
3. One trained on 50 authors.

Each of these models was tested on a dataset containing only comments from authors it was trained on (henceforth referred to as "n," where n is the number of known authors), and on a dataset containing comments from authors it knows, as well as 5 unknown authors (henceforth referred to as "$n/5$").

We also tried performing authorship verification by computing the Euclidean distance between the probability distributions generated by each model from the input comments, and returning a result based on whether the value exceeded a threshold, similar to the deep learning approach.

For the larger models (20 authors, 50 authors), we drastically reduced the number of training examples to a constant 16,000 to see if the reduced dataset size would have a small enough effect on zero-shot performance to justify faster training.

3.1.2 Known Author Performance

Table 1: Comparison of Models by Time Cost and Performance

	Training Time	Testing Time	Attribution Accuracy	Verification Accuracy
5-Author Model	10 minutes	4 minutes	31.0%	22.4%
20-Author Model	140 minutes	40 minutes	10.9%	5.9%
50-Author Model	716 minutes	172 minutes	5.6%	3.8%

3.1.3 Zero-Shot Performance

Table 2: Comparison of Models by Time Cost and Zero-Shot Performance

	Train Time (16k Examples)	AA Accuracy	AV Accuracy	AV (Full Model)	AV Accuracy (PMF Comp)
5/5	N/A	20.0%	12.1%	N/A	11.6%
20/5	40 minutes	9.9%	4.7%	4.7%	4.8%
50/5	77 minutes	5.4%	2.1%	2.2%	2.2%

3.1.4 Interpretations

Evidently, the statistical learning approach provides significantly worse results than the deep learning approach. Though it produces results comparable to [5] on much shorter training samples, on the comparatively less performant TF-IDF rather than n-grams. However, it is much faster for both training and testing, allowing for more rapid iteration, especially when the training data is reduced. In that regard, we found that including 80% of all possible data from each commenter leads to overfitting, and reducing the dataset to 16,000 training examples provides nearly equivalent results with significantly less training overhead.

3.2. Deep Learning Approach

3.2.1 Experimentation Explanation

We trained three models:

1. One trained on 5 authors,
2. One trained on 20 authors,
3. One trained on 50 authors.

A subset of the authors from the entire dataset was chosen randomly, and then two subsets of their messages were selected for training and testing. In addition, 5 unrelated authors were chosen to test the model's ability on authors it had never seen before.

3.2.2 Interpretation

Accuracies

First, consider the expected results (Table 3). As the number of authors increased, it became more challenging for the model to classify them correctly. However, all models achieved performance significantly above random guessing and notably better than their pure SVM counterparts. This pattern persists across all verification methods. This behavior is expected for both Predicted Author and Logits Verification, as they rely on the model's classification ability. Although Embedding Verification consistently produced the highest scores across all models, its performance still declined as the number of authors to verify increased.

When evaluating accuracies on unknown authors, direct comparison with known authors is challenging because there were always five unknown authors, even for models with 20 and 50 known authors. Overall, accuracy increased with embedding models trained on more authors, but the improvement was not dramatic.

The best overall performance was observed with the embedding towers trained using 20 authors. This result

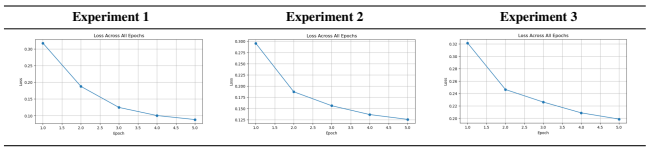
Table 3: Accuracy Metrics for Experiments

	Experiment 1 (5 Known)		Experiment 2 (20 Known)		Experiment 3 (50 Known)	
Metric	Known	Unknown	Known	Unknown	Known	Unknown
Classification	91.25	70.88	56.81	89.62	71.56	74.25
Embedding Verification	91.74	76.50	81.38	77.00	79.62	77.75
Predicted Author Verification	90.38	78.00	72.62	90.20	70.12	75.25
Logits Verification	84.00	73.00	57.75	83.25	55.12	66.75

may imply several factors, including that the 50-author model may have been undertrained. Since we used the same amount of training data and training time for each model, models with a higher number of authors effectively received less training data per author.

In summary, all models generalized well to unseen authors, indicating that the embedding space was not overfitted to the specific authors seen during training. A highlight is the 90% verification accuracy on unknown authors by the 20-author model, which is comparable to the accuracy of the 5-author model on known authors.

Table 4: Loss Graphs for Each Experiment



Loss

The loss decreases with each successive epoch, as seen in table 4, indicating that the model is training effectively. Two points should be noted: first, since the loss is computed solely for training the embedding towers, it is not directly correlated with the overall performance of the complete model (which includes the SVM classifier on top of the embedding tower). Second, the graph shows that the loss has not completely plateaued, suggesting that additional training might further improve performance. However, since our goal is to develop a generalized model that performs well on unseen authors, early stopping might actually help reduce overfitting. This represents a potential avenue for further research.

Embeddings

Tables 5 and 6 illustrate the embedding space generated by training the contrastive tower. The embeddings are visualized using t-SNE, which is known to better preserve relational information when reducing high-dimensional

Table 5: Pre- and Post-Training Embedding Graphs on Known Authors' Messages

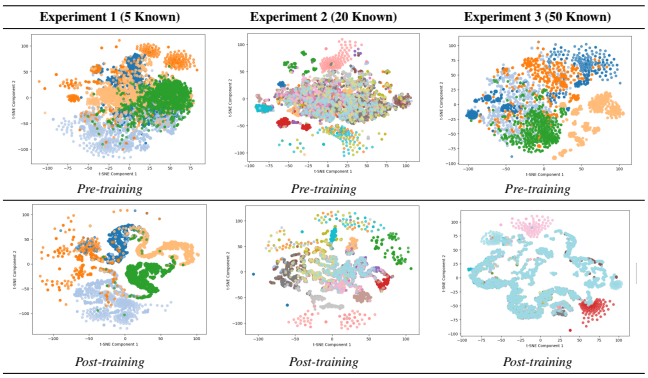
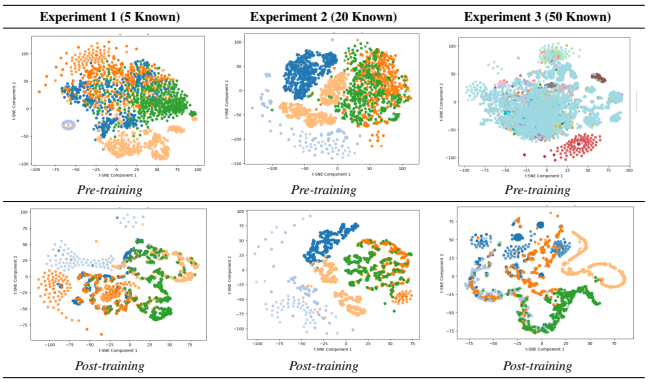


Table 6: Pre- and Post-Training Embedding Graphs on 5 Unknown Authors' Messages



data to 2D. The top row in each figure shows the embeddings of messages before any fine-tuning and training. Notably, there is already a good degree of separation, implying that the BERT encoder alone performs well in distinguishing the data.

The second row shows the embeddings after training; here, the data is more distinctly separated, and different geometric patterns begin to emerge.

Figure 1 displays the embeddings of test messages from

authors on which the contrastive tower was trained (i.e., the tower has seen these authors before, though the messages are new). In contrast, Figure 2 presents the embeddings when the tower is trained on one set of authors but tested on completely different authors and messages.

4. Conclusion

All in all, we found that our hypothesis was correct. The statistical models struggled to produce meaningful results beyond the naive approach, while the deep learning approach achieved higher accuracy and maintained generalizability to unknown authors. However, the statistical approach uniquely could be trained much faster than the deep learning approach, and it could also perform prediction much faster as well.

On the other hand, the deep learning models performed significantly better than the statistical models in both known-author and zero-shot performance, never dipping below 55% for any category. It even performed zero-shot verification to a very acceptable degree, of around 70% acccuracy. This model evidently generalizes very well to unknown authors, unlike existing state-of-the-art models for authorship verification. While statistical models can perform well on longer texts as shown in [5], they evidently do not scale well to texts as short as Reddit comments.

Our current recommendation for model implementation is that the statistical model be used for more rapid iteration, and as a stopgap measure until the deep learning model is fully trained and deployed.

References

[1] Amirah Almutairi, BooJoong Kang, and Nawfal Al Hashimy. "BiBERT-AV: Enhancing Authorship Verification Through Siamese Networks with Pre-trained BERT and Bi-LSTM". In: *Ubiquitous Security*. Ed. by Guojun Wang et al. Singapore: Springer Nature Singapore, 2024, pp. 17–30. ISBN: 978-981-97-1274-8.

[2] David Curry. *Reddit Revenue and Usage Statistics (2025)*. March 19, 2025. 2025. URL: https://www.businessofapps.com/data/reddit-statistics/.

[3] Baixiang Huang, Canyu Chen, and Kai Shu. "Can Large Language Models Identify Authorship?" In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. Ed. by Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 445–460. DOI: 10.18653/v1/2024.findings-emnlp.26. URL: https://aclanthology.org/2024.findings-emnlp.26/.

[4] Momen Ibrahim et al. "Enhancing Authorship Verification using Sentence-Transformers". In: *Notebook for PAN at CLEF 2023*. CEUR Workshop Proceedings, CEUR-WS.org, licensed under CC BY 4.0. CLEF 2023 – Conference and Labs of the Evaluation Forum. Thessaloniki, Greece, 2023. URL: https://github.com/RanaAyman/AV.

[5] Feristah Orucu and Gökhan Dalkılıç. "Author identification Using N-grams and SVM". In: June 2010.